

Δομές Δεδομένων

1^η Εργασία

2025-2026

Στα πλαίσια της εργασίας ζητείται να συγγράψετε πρόγραμμα σε C++ το οποίο θα λειτουργεί ως σύστημα λογισμικού για τη διαχείριση βιβλίων και των αντιτύπων τους σε μια βιβλιοθήκη. **Στα πλαίσια της εργασίας θα πρέπει να χρησιμοποιήσετε την συνδεδεμένη αναπαράσταση για τις γραμμικές λίστες.** Για λεπτομέρειες δείτε το Κεφ. 3.4 του βιβλίου “Δομές Δεδομένων, Αλγόριθμοι και Εφαρμογές στη C++”. **Δεν επιτρέπεται η χρήση των έτοιμων δομών δεδομένων που προσφέρει η C++ (C++ Containers), ούτε η χρήση πινάκων για την πραγματοποίηση των λειτουργιών που περιγράφονται παρακάτω και η μετατροπή στην συνέχεια σε λίστα.**

Σκοπός του προγράμματος που θα φτιάξετε είναι να δημιουργήσετε μια «λίστα από λίστες». Συγκεκριμένα, το πρόγραμμα θα διαβάζει από ένα αρχείο (“library.txt”) πληροφορίες για κάθε βιβλίο που υπάρχει διαθέσιμο στην βιβλιοθήκη. Κάθε βιβλίο θα αναπαρίσταται από ένα αντικείμενο της κλάσης BookNode, τα οποία θα αποτελούν τους κόμβους της λίστας LOB (List Of Books). Κάθε αντίτυπο ενός βιβλίου θα αναπαρίσταται από ένα αντικείμενο της κλάσης CopyNode. Η κλάση BookNode θα πρέπει να περιλαμβάνει τα εξής πεδία:

- Το αλφαριθμητικό title, στο οποίο θα αποθηκεύεται ο τίτλος του βιβλίου.
- Το αλφαριθμητικό author, στο οποίο θα αποθηκεύεται το ονοματεπώνυμο του συγγραφέα.
- Το αλφαριθμητικό ISBN, στο οποίο θα αποθηκεύεται ο κωδικός ISBN του βιβλίου.
- Τον δείκτη copiesHead προς μια λίστα LOC (List Of Copies), στην οποία θα αποθηκεύεται πληροφορία για κάθε ένα από τα διαθέσιμα αντίτυπα του βιβλίου στην βιβλιοθήκη.
- Τον δείκτη next προς το επόμενο στοιχείο της λίστας LOB.

Για κάθε βιβλίο που διαβάζει το πρόγραμμα από το αρχείο **θα πρέπει να δημιουργεί ένα νέο αντικείμενο της κλάσης BookNode και να το τοποθετεί στο σωστό σημείο της λίστας LOB, ώστε αυτή να διατηρείται ταξινομημένη με βάση τον τίτλο του βιβλίου.** Η σύγκριση τίτλων μπορεί να γίνει με την τελεστική σύγκριση (<) των συμβολοσειρών της C++. Η εισαγωγή στην σωστή θέση θα πρέπει να γίνεται μόλις δημιουργηθεί ο νέος κόμβος. **Δεν επιτρέπεται η δημιουργία όλης της λίστας LOB και η ταξινόμηση της στο τέλος.** Θεωρείστε πως κάθε τίτλος βιβλίου στο αρχείο είναι μοναδικός.

Στην συνέχεια, **θα πρέπει να δημιουργεί ένα αντικείμενο της κλάσης CopyNode για κάθε αντίτυπο του βιβλίου και να το τοποθετεί στο σωστό σημείο της λίστας LOC, ώστε αυτή να διατηρείται ταξινομημένη με βάση τον αριθμό αντιτύπου.** Η εισαγωγή στην σωστή θέση θα πρέπει να γίνεται μόλις δημιουργηθεί ο νέος κόμβος. **Δεν επιτρέπεται η δημιουργία όλης της λίστας LOC και η ταξινόμηση της στο τέλος.** Η κλάση CopyNode πρέπει να περιλαμβάνει τα εξής πεδία:

- Τον ακέραιο αριθμό copyID, στον οποίο θα αποθηκεύεται ο αριθμός αντιτύπου ενός βιβλίου.
- Το αλφαριθμητικό status, στο οποίο θα αποθηκεύεται η κατάσταση του αντιτύπου. Το πεδίο θα πρέπει να παίρνει τιμές από το σύνολο {“available”, “borrowed”, “damaged”}.
- Τον δείκτη next προς το επόμενο στοιχείο της λίστας LOC.

Τέλος, η κλάση Library θα περιέχει το πεδίο booksHead, που θα είναι ένας δείκτης προς την λίστα LOB.

Το σχήμα στην τελευταία σελίδα της εκφώνησης δίνεται προς διευκρίνιση λεπτομερειών. Για παράδειγμα, υποθέστε πως κατά την διάρκεια ανάγνωσης του αρχείου βρέθηκε βιβλίο με τίτλο “Great Book of Data Structures”. Θα πρέπει να κατασκευαστεί ο αντίστοιχος κόμβος τύπου BookNode και αυτός θα πρέπει να τοποθετηθεί μετά το βιβλίο με τίτλο “Fundamentals of Data Structures” (όπως δείχνει το κόκκινο βέλος), ώστε η λίστα LOB να παραμείνει ταξινομημένη. Επειδή το βιβλίο έχει δύο αντίτυπα θα πρέπει να δημιουργηθούν και οι αντίστοιχοι δύο κόμβοι τύπου CopyNode της λίστας LOC. Προσέξτε πως και η λίστα αυτή είναι ταξινομημένη για κάθε βιβλίο (κόμβο τύπου BookNode) με βάση τον αριθμό αντιτύπου.

Κυρίως λειτουργία προγράμματος

Κατά την εκκίνηση του προγράμματος, αυτό θα διαβάζει το αρχείο εισόδου με όνομα "library.txt" και θα δημιουργεί τις λίστες, όπως περιγράφηκε προηγουμένως. Στην συνέχεια, το πρόγραμμα θα εισέρχεται σε έναν βρόχο (loop), στο οποίο θα εμφανίζεται ένα menu επιλογών. Το πρόγραμμα θα τερματίζει όταν ο χρήστης επιλέξει το 0 ("Έξοδος"). Οι επιλογές του μενού θα είναι οι παρακάτω:

```
===== ΒΙΒΛΙΟΘΗΚΗ - ΚΕΝΤΡΙΚΟ MENU =====
1. Φόρτωση βιβλιοθήκης από αρχείο
2. Αποθήκευση βιβλιοθήκης σε αρχείο
3. Εμφάνιση όλων των βιβλίων
4. Αναζήτηση βιβλίου (με ISBN)
5. Προσθήκη νέου βιβλίου
6. Διαγραφή βιβλίου
7. Προσθήκη αντιτύπου σε βιβλίο
8. Διαγραφή αντιτύπου από βιβλίο
9. Δανεισμός αντιτύπου
10. Επιστροφή αντιτύπου
11. Εμφάνιση διαθέσιμων αντιτύπων βιβλίου
0. Έξοδος
=====
```

Η λειτουργία των επιλογών του προγράμματος και οι εξαρτήσεις μεταξύ τους ακολουθεί:

Επιλογή 1 — Φόρτωση από αρχείο

Διαβάζει το αρχείο "library.txt" και δημιουργεί εκ νέου τη δομή στη μνήμη. Αν υπάρχει ήδη φορτωμένη βιβλιοθήκη, ζητά επιβεβαίωση από τον χρήστη πριν αντικαταστήσει τα δεδομένα. Καλεί εσωτερικά τον καταστροφέα για να απελευθερωθεί η προηγούμενη μνήμη.

Επιλογή 2 — Αποθήκευση σε αρχείο

Προϋπόθεση: να υπάρχει φορτωμένη βιβλιοθήκη. Αποθηκεύει την τρέχουσα κατάσταση στο αρχείο "library.txt" στην ίδια μορφή που την διαβάζει η επιλογή 1.

Επιλογή 3 — Εμφάνιση όλων

Προϋπόθεση: να υπάρχει τουλάχιστον ένα βιβλίο. Διατρέχει την κύρια λίστα και για κάθε βιβλίο εμφανίζει τίτλο, συγγραφέα, ISBN και αριθμό διαθέσιμων / συνολικών αντιτύπων.

Επιλογή 4 — Αναζήτηση βιβλίου

Αποκλεισμός: δεν τροποποιεί τίποτα. Ζητά ISBN και εμφανίζει τα στοιχεία του βιβλίου καθώς και λεπτομερή λίστα όλων των αντιτύπων με κατάστασή τους.

Επιλογή 5 — Προσθήκη βιβλίου

Αποκλεισμός: απορρίπτει εισαγωγή αν το ISBN υπάρχει ήδη. Ζητά τίτλο, συγγραφέα και ISBN. Το νέο βιβλίο προστίθεται στο κατάλληλο σημείο της λίστας LOB, χωρίς όμως αντίτυπα.

Επιλογή 6 — Διαγραφή βιβλίου

Αποκλεισμός: απαγορεύεται αν οποιοδήποτε αντίτυπο έχει κατάσταση "borrowed". Διαφορετικά, απελευθερώνεται όλη η μνήμη: πρώτα η εσωτερική λίστα αντιτύπων, μετά ο κόμβος του βιβλίου.

Επιλογή 7 — Προσθήκη αντιτύπου

Εξάρτηση: το βιβλίο πρέπει να υπάρχει. **Αποκλεισμός:** ο αριθμός αντιτύπου δεν πρέπει να χρησιμοποιείται ήδη. Ζητείται πρώτα το ISBN για να βρεθεί και να επιβεβαιωθεί πως το βιβλίο υπάρχει. Το νέο αντίτυπο αρχικοποιείται πάντα ως "available".

Επιλογή 8 — Διαγραφή αντιτύπου

Αποκλεισμός: απαγορεύεται αν το αντίτυπο είναι "borrowed". Αν μετά την διαγραφή ένα βιβλίο δεν έχει πλέον αντίτυπα, παραμένει στη βιβλιοθήκη μέχρι να διαγραφεί μέσω της επιλογής 6. Ζητείται πρώτα το ISBN για να βρεθεί και να επιβεβαιωθεί πως το βιβλίο υπάρχει.

Επιλογή 9 — Δανεισμός αντιτύπου

Προϋπόθεση: το αντίτυπο πρέπει να είναι "available". **Αποκλεισμός:** απαγορεύεται δανεισμός αντιτύπου με κατάσταση "borrowed" ή "damaged". Μετά την επιτυχή εκτέλεση εμφανίζεται ο ενημερωμένος αριθμός διαθέσιμων αντιτύπων. Ζητείται πρώτα το ISBN για να βρεθεί και να επιβεβαιωθεί πως το βιβλίο υπάρχει.

Επιλογή 10 — Επιστροφή αντιτύπου

Προϋπόθεση: το αντίτυπο πρέπει να είναι "borrowed". Κατά την επιστροφή ρωτά τον χρήστη αν το αντίτυπο επιστράφηκε σε καλή κατάσταση. Αν όχι, η νέα κατάσταση γίνεται "damaged" αντί "available". Ζητείται πρώτα το ISBN για να βρεθεί και να επιβεβαιωθεί πως το βιβλίο υπάρχει.

Επιλογή 11 — Διαθέσιμα αντίτυπα

Αποκλεισμός: δεν τροποποιεί τίποτα. Ζητά ISBN και εμφανίζει μόνο τα αντίτυπα με κατάσταση "available" και τον συνολικό αριθμό τους.

Επιλογή 0 — Έξοδος

Ρωτά αν ο χρήστης θέλει να αποθηκεύσει πριν κλείσει. Αν ναι, εκτελεί αυτόματα την επιλογή 2. Στη συνέχεια απελευθερώνεται όλη η μνήμη μέσω του καταστροφέα.

Στον παρακάτω πίνακα παρουσιάζονται συγκεντρωτικά οι παραπάνω απαιτήσεις και προϋποθέσεις:

Επιλογή	Απαιτεί	Αποκλείεται αν
2 — Αποθήκευση	Βιβλιοθήκη φορτωμένη	—
5 — Νέο βιβλίο	—	ISBN ήδη υπάρχει στη λίστα
6 — Διαγραφή βιβλίου	Βιβλίο με το ISBN να υπάρχει	Υπάρχει αντίτυπο με status=borrowed
7 — Νέο αντίτυπο	Βιβλίο να υπάρχει στη λίστα	copyId ήδη υπάρχει στο βιβλίο
8 — Διαγραφή αντιτύπου	Αντίτυπο να υπάρχει	status=borrowed ή μοναδικό αντίτυπο
9 — Δανεισμός	Αντίτυπο να υπάρχει	status ≠ available
10 — Επιστροφή	Αντίτυπο να υπάρχει	status ≠ borrowed

Λειτουργίες κλάσεων

Για την υλοποίηση των επιλογών του menu υλοποιήστε τις παρακάτω λειτουργίες στις αντίστοιχες κλάσεις:

1. Λειτουργίες Κλάσης Library (λίστα LOB)
 - a. addBook(title, author, isbn) — Προσθήκη νέου βιβλίου στο κατάλληλο σημείο της λίστας LOB.
 - b. removeBook(isbn) — Διαγραφή βιβλίου βάσει ISBN (απελευθέρωση όλης της μνήμης, συμπεριλαμβανομένης της εσωτερικής λίστας αντιτύπων).
 - c. findBook(isbn) — Αναζήτηση βιβλίου βάσει ISBN. Επιστρέφει δείκτη στον κόμβο ή nullptr αν δεν βρεθεί.
 - d. printAll() — Εκτύπωση όλων των βιβλίων (τίτλος, συγγραφέας, ISBN, αριθμός διαθέσιμων/συνολικών αντιτύπων).
 - e. loadFromFile(filename) — Φόρτωση βιβλιοθήκης από αρχείο.
 - f. saveToFile(filename) — Αποθήκευση βιβλιοθήκης σε αρχείο (ίδια μορφή με το αρχείο εισόδου).
 - g. ~Library() — Καταστροφές που απελευθερώνει όλη τη δυναμική μνήμη.
2. Λειτουργίες Κλάσης BookNode (λίστα LOC)
 - a. addCopy(copyId, status="available") — Προσθήκη νέου αντιτύπου με εξ ορισμού κατάσταση "available".
 - b. removeCopy(copyId) — Διαγραφή αντιτύπου βάσει copyId.
 - c. borrowCopy(copyId) — Αλλαγή κατάστασης αντιτύπου σε "borrowed". Επιστρέφει false αν δεν είναι διαθέσιμο.
 - d. returnCopy(copyId) — Αλλαγή κατάστασης αντιτύπου σε "available" ή "damaged".
 - e. countAvailable() — Επιστροφή του αριθμού των διαθέσιμων αντιτύπων.

Δημιουργία αρχείου εισόδου "library.txt"

Προς διευκόλυνση σας, δίνεται πρόγραμμα σε Python που δημιουργεί το αρχείο εισόδου "library.txt". Μπορείτε να αλλάξετε το πλήθος των βιβλίων που θα περιέχει το αρχείο τροποποιώντας την τιμή της μεταβλητής NUM_BOOKS. Χρησιμοποιήστε αρχικά μικρότερες τιμές για να μπορείτε να ελέγχετε την ορθότητα του προγράμματος σας ως προς την δημιουργία των λιστών και τις λειτουργίες του menu.

Ανάγνωση αρχείου εισόδου "library.txt"

Προς διευκόλυνση σας, παρέχεται έτοιμη η συνάρτηση parseLine(), η οποία διαβάζει μια γραμμή κειμένου από το αρχείο "library.txt", την διαχωρίζει στα tokens που την αποτελούν (με τον χαρακτήρα '|' ως διαχωριστικό) και επιστρέφει τα tokens σε ένα αντικείμενο τύπου vector<string>. Δίνεται επίσης ένας σκελετός (skeleton) για την συνάρτηση loadFromFile(). Θα πρέπει να συμπληρώσετε τα δύο TODO blocks στη loadFromFile() ώστε να δημιουργείτε και να εισάγετε τους κόμβους στις λίστες. Ο σκοπός της εργασίας είναι η διαχείριση δομών δεδομένων και όχι το parsing αρχείων. Για τον λόγο αυτό σας παρέχονται αυτές οι συναρτήσεις.

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

// Διαχωρίζει μια γραμμή στα tokens της βάσει του διαχωριστικού delim.
// Παράδειγμα εισόδου : "BOOK|978-960-01-0001-1|Αλγόριθμοι|Cormen"
// Παράδειγμα εξόδου : {"BOOK", "978-960-01-0001-1", "Αλγόριθμοι", "Cormen"}
std::vector<std::string> parseLine(const std::string& line, char delim = '|') {
    std::vector<std::string> tokens;
    std::stringstream ss(line);
    std::string token;
    while (std::getline(ss, token, delim)) {
        tokens.push_back(token);
    }
    return tokens;
}
```

```

// Φορτώνει τη βιβλιοθήκη από αρχείο.
// ΔΕΝ παρέχεται έτοιμη – πρέπει να την υλοποιήσετε εσείς.
// Το skeleton σάς δείχνει πώς να χρησιμοποιήσετε την parseLine().
void Library::loadFromFile(const std::string& filename) {
    std::ifstream file(filename);
    if (!file.is_open()) {
        std::cout << "Σφάλμα: δεν ήταν δυνατό το άνοιγμα του αρχείου '"<<filename<<"'\n";
        return;
    }

    std::string line;
    while (std::getline(file, line)) {
        if (line.empty()) continue; // παράλειψη κενών γραμμών

        std::vector<std::string> tokens = parseLine(line);

        if (tokens[0] == "BOOK") {
            // tokens[1] = ISBN
            // tokens[2] = τίτλος
            // tokens[3] = συγγραφέας
            //
            // TODO: Δημιουργήστε έναν νέο BookNode και
            //       εισάγετέ τον στην κύρια λίστα.

        } else if (tokens[0] == "COPY") {
            // tokens[1] = copyId (μετατροπή: std::stoi(tokens[1]))
            // tokens[2] = status
            //
            // TODO: Βρείτε τον τελευταίο BookNode που εισήχθη
            //       και προσθέστε σε αυτόν το αντίτυπο.

        } else {
            std::cout << "Άγνωστη εγγραφή: " << tokens[0] << "\n";
        }
    }

    file.close();
}

```

Παραδοτέα

Θα πρέπει να παραδοθεί ο πηγαίος κώδικας μαζί με τον εκτελέσιμο. Ιδιαίτερη βαρύτητα θα πρέπει να δοθεί στη σωστή τεκμηρίωση των προγραμμάτων σας. Θα πρέπει λοιπόν ο κώδικας σας να συνοδεύεται από ξεχωριστό κείμενο που θα παρέχει λεπτομερή περιγραφή των τεχνικών σας. Επίσης, εντός του πηγαίου κώδικα θα πρέπει να συμπεριληφθούν «πυκνά» **σχόλια ουσίας**. Η παράδοση των εργασιών θα γίνει μέσω του Thales.

Η εργασία μπορεί να εκπονηθεί από **ομάδα μέχρι δύο ατόμων αυστηρώς**.

Προθεσμία Παράδοσης

Πέμπτη 30 Απριλίου 2026

